Digital Electronic Circuits (EC2L004)

Lecture 16: Registers

Learning Objectives

Registers

- Introduction: Registers
  - Simple Registers
  - Registers with Parallel Load
- Using Registers to implement Sequential Circuits
- Shift Registers
  - Serial In/Serial Out Shift Registers
  - Serial In/Parallel Out Shift Registers
  - Parallel In/Serial Out Shift Registers
  - Parallel In/Parallel Out Shift Registers

Learning Objectives

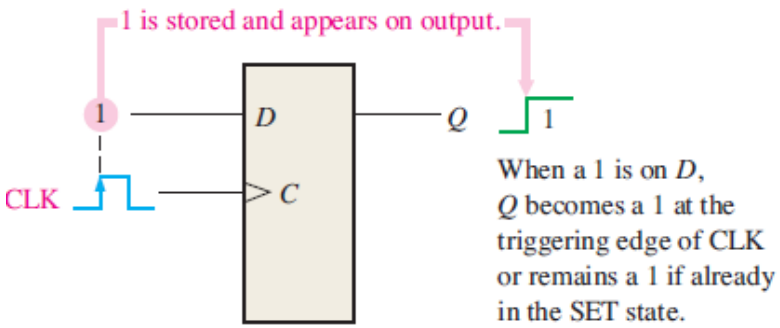
- Bidirectional Shift Registers
- An Application – Serial Addition
- Shift Register Counters
  - Ring Counters
  - Johnson Counters

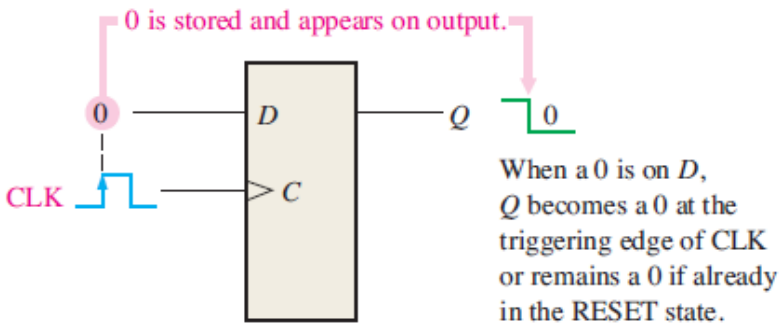
Introduction: Registers

- A register is a digital circuit with two basic functions: data storage and data movement.

- The storage capability of a register makes it an important type of memory device.

- A register can consist of one or more flip-flops used to store and shift data.





Introduction: Registers

- An *n-bit register* has a group of *n* flip-flops and some logic gates and can store *n* bits of information.

- The flip-flops store the information while the gates control when and how new information is transferred into the register.

- Some functions of register:

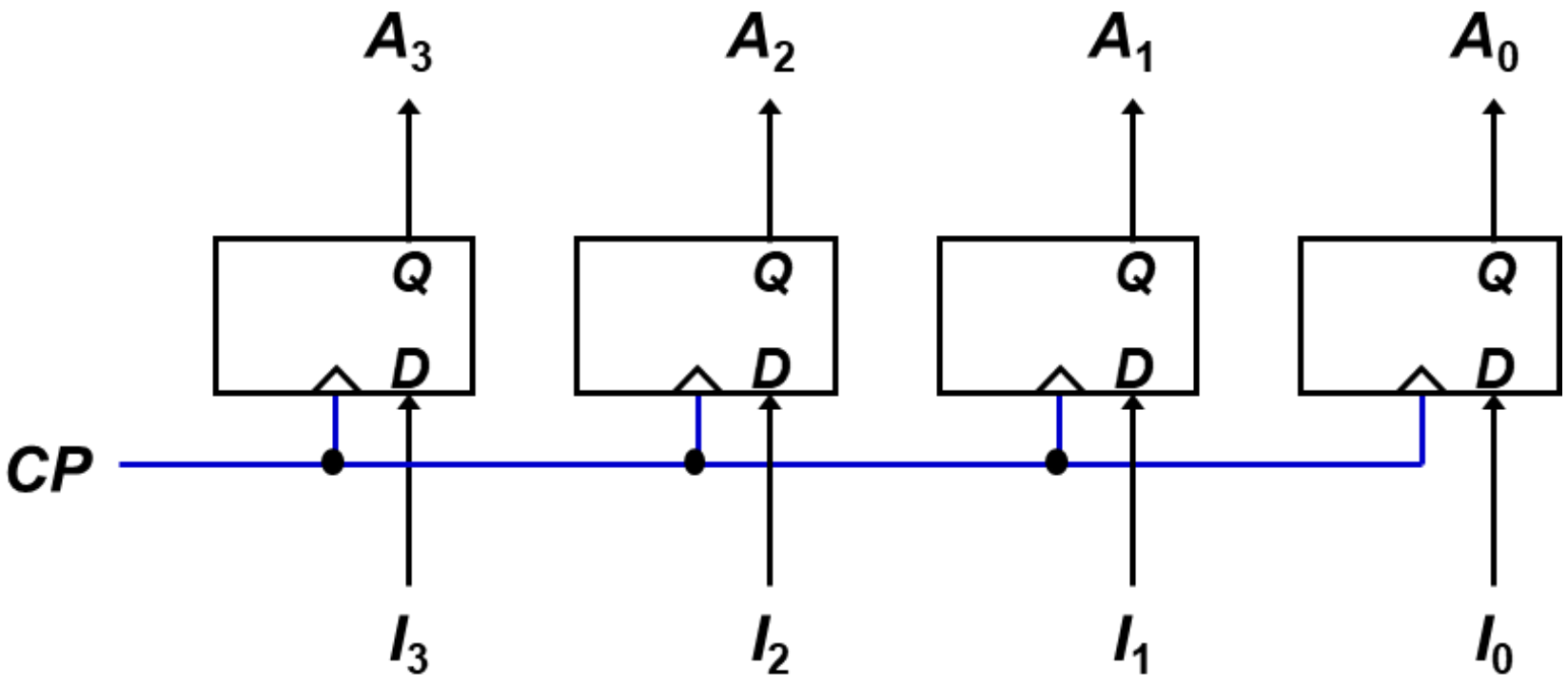
  - Retrieve data from register

  - Store/load new data into register (serial or parallel)

  - Shift the data within register (left or right)

Simple Registers

- No external gates.
- Example: A 4-bit register. A new 4-bit data is loaded every clock cycle.



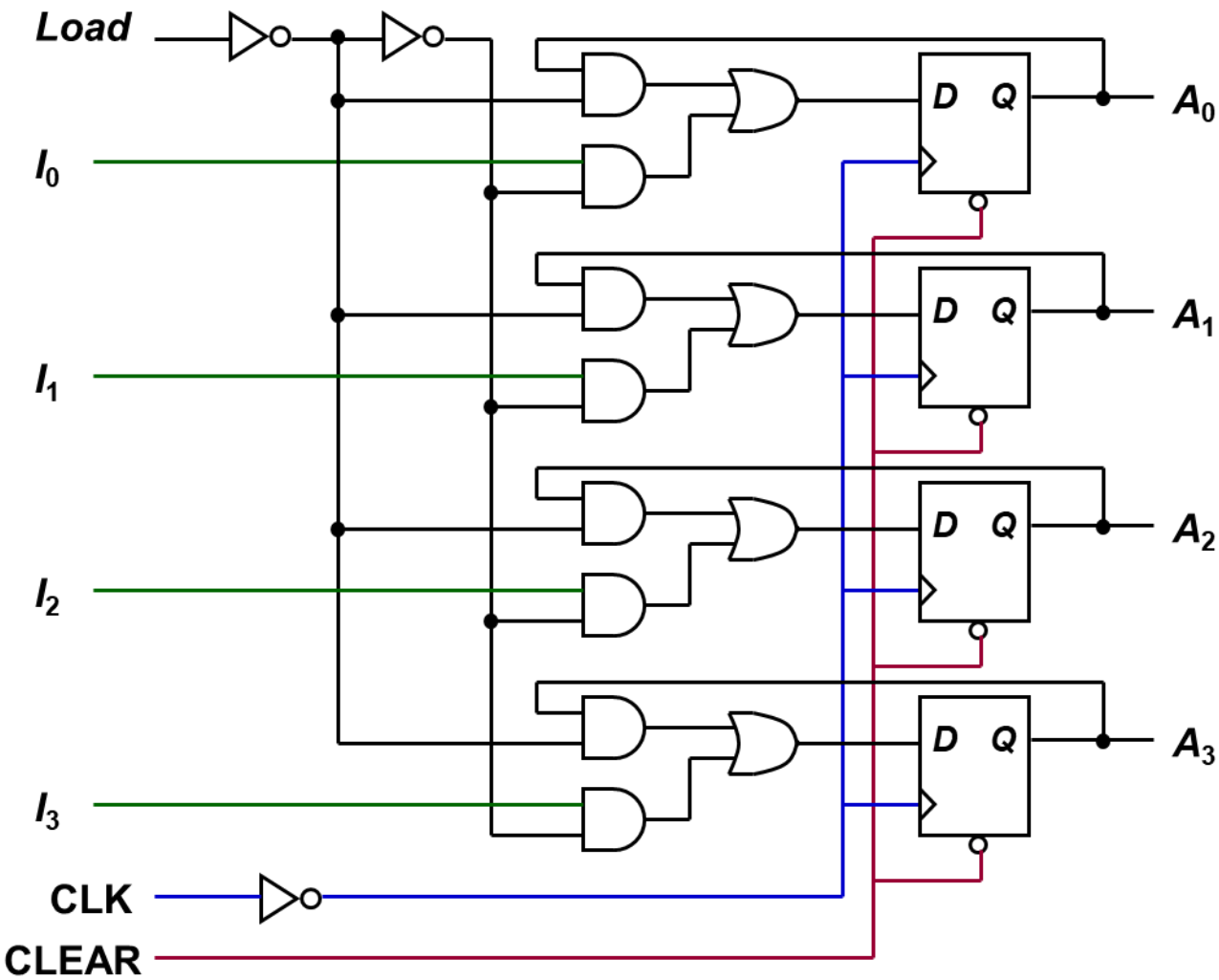
Registers With Parallel Load

- Instead of loading the register at every clock pulse, we may want to control when to load.

- Loading a register: transfer new information into the register. Requires a load control input.

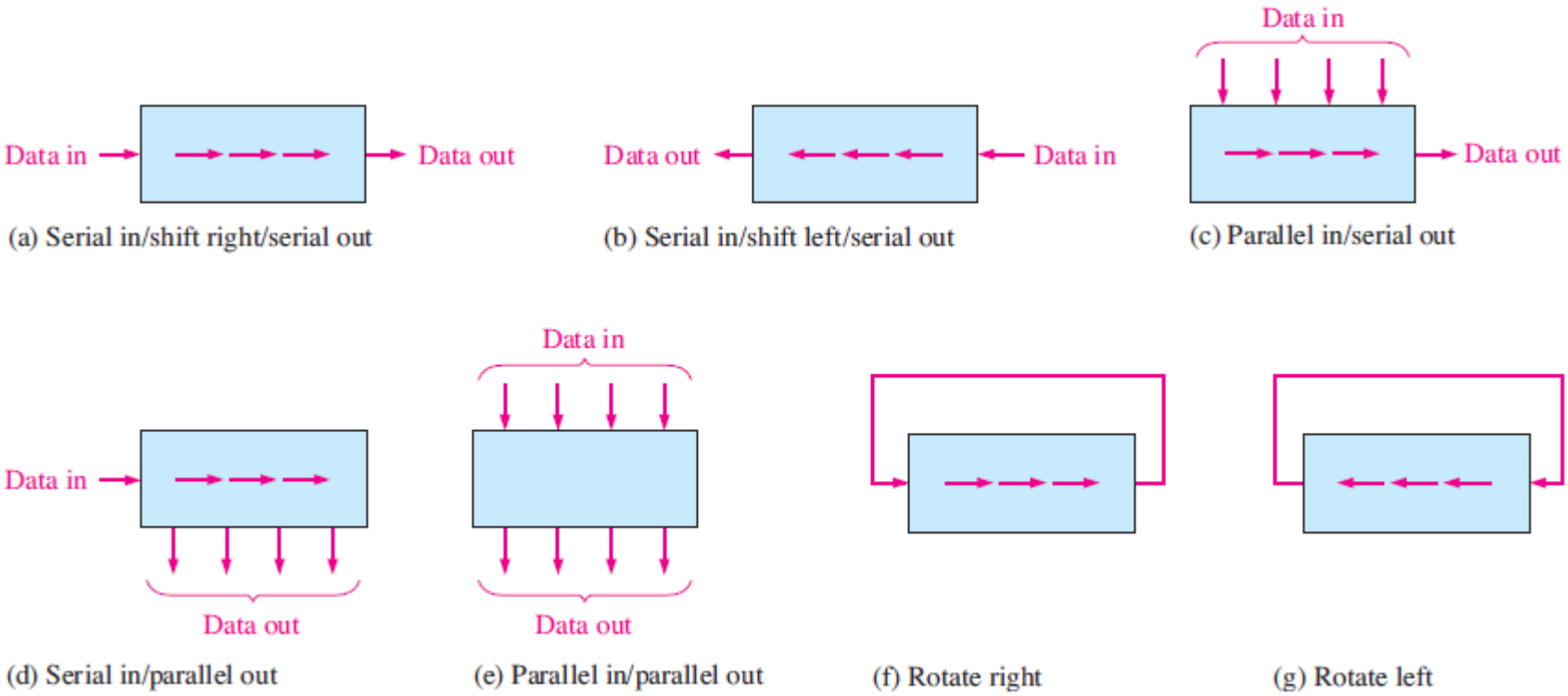
- Parallel loading: all bits are loaded simultaneously.

Registers With Parallel Load



Shift Registers

- Basic data movement in shift registers



Serial In/Serial Out Shift Registers

- Accepts data serially – one bit at a time – and also produces output serially.

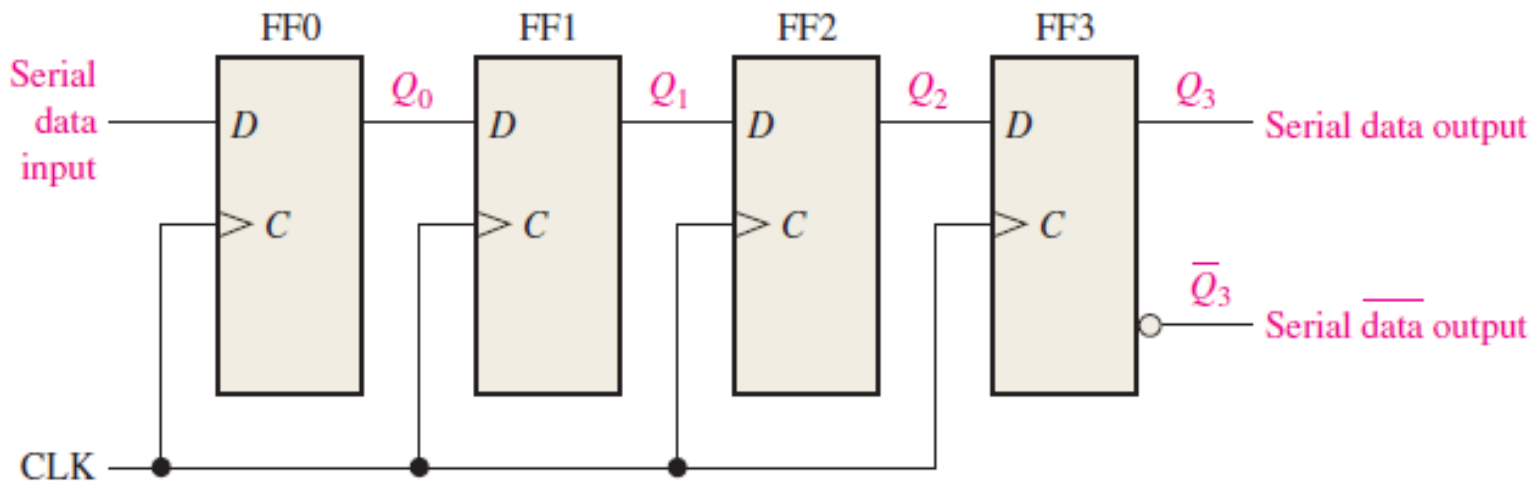
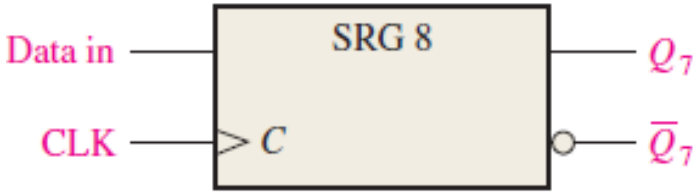
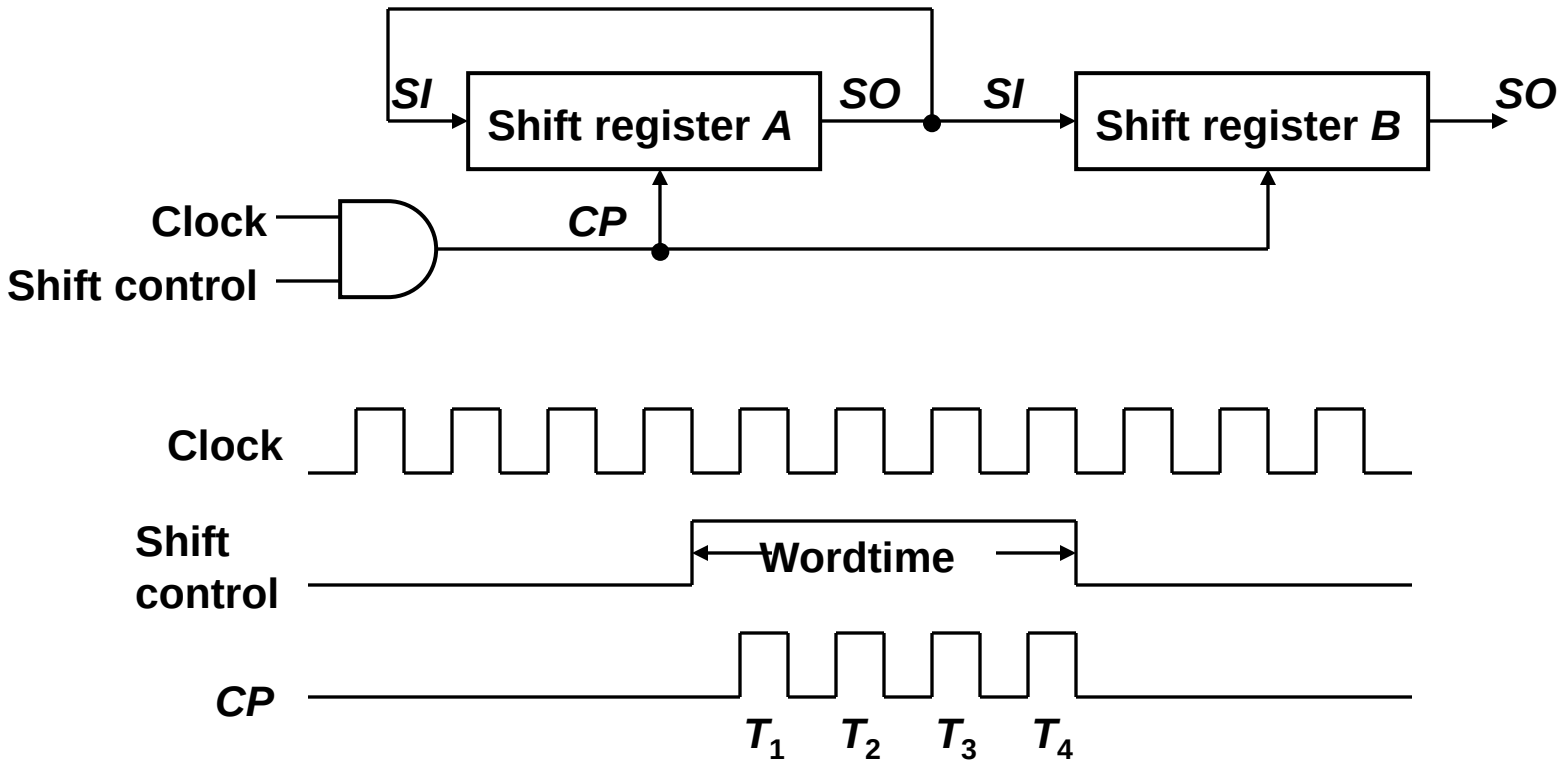


FIGURE 8–3 Serial in/serial out shift register.



Serial In/Serial Out Shift Registers

- Application: Serial transfer of data from one register to another.



Serial In/Serial Out Shift Registers

- Serial-transfer example.

Timing Pulse	Shift register *A*				Shift register *B*				Serial output of *B*
Initial value	1	0	1	1	0	0	1	0	0
After T_1	1	1	0	1	1	0	0	1	1
After T_2	1	1	1	0	1	1	0	0	0
After T_3	0	1	1	1	0	1	1	0	0
After T_4	1	0	1	1	1	0	1	1	1

Serial In/Parallel Out Shift Registers

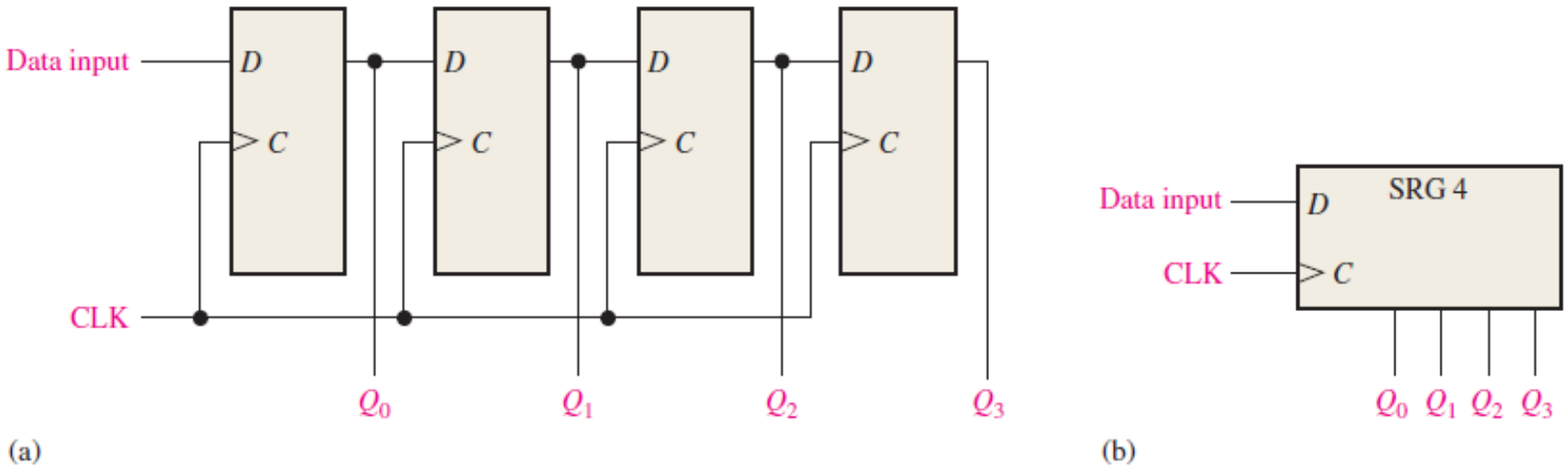
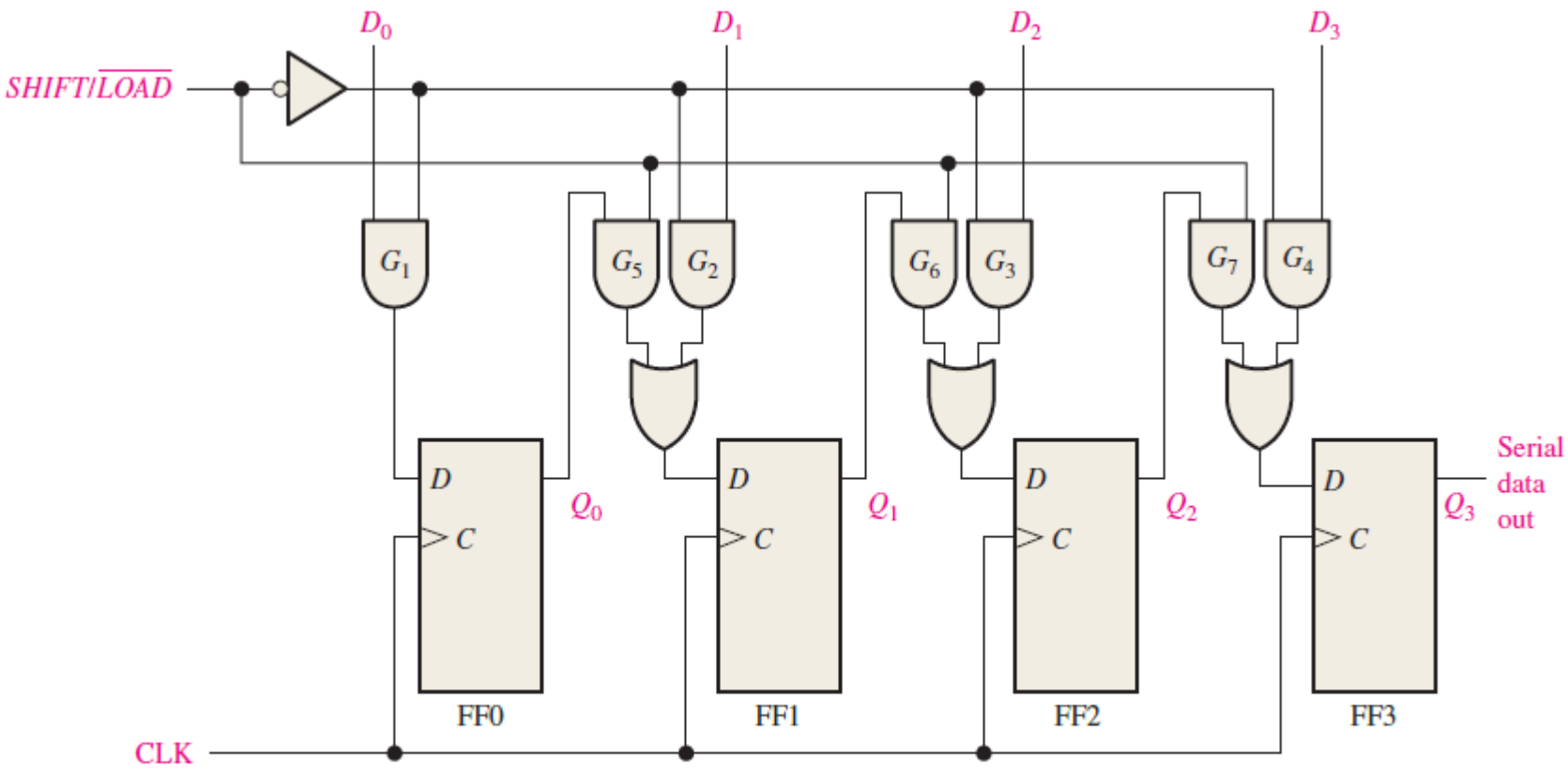


FIGURE 8–6 A serial in/parallel out shift register.

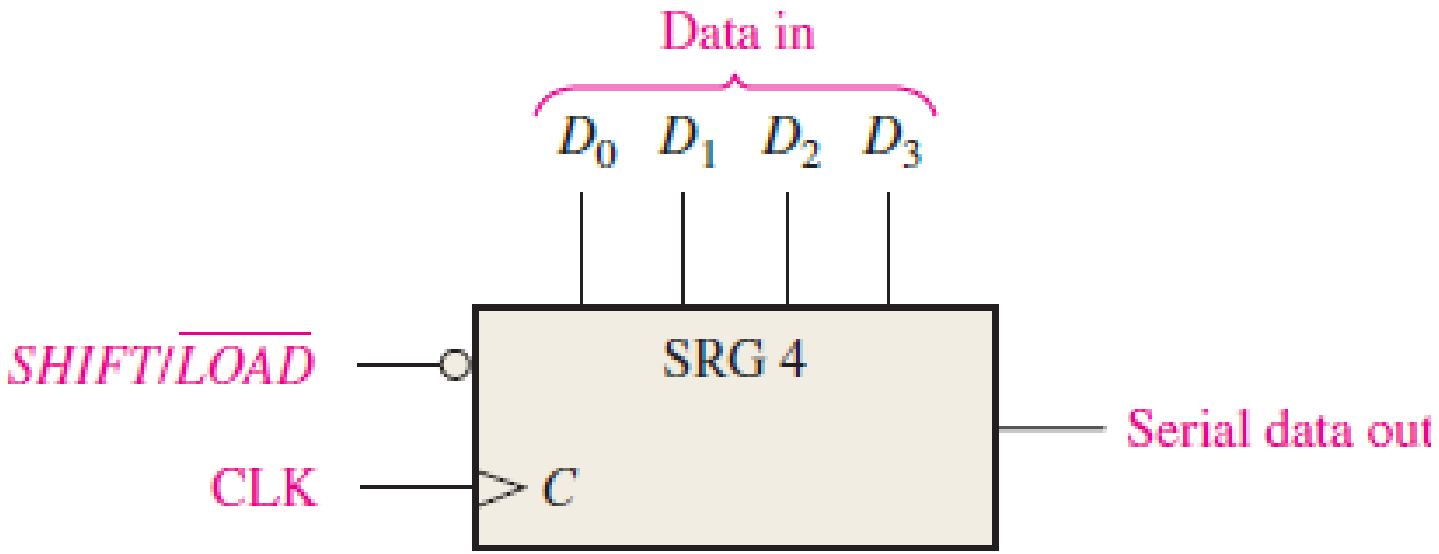
Parallel In/Serial Out Shift Registers

- Bits are entered simultaneously, but output is serial.



(a) Logic diagram

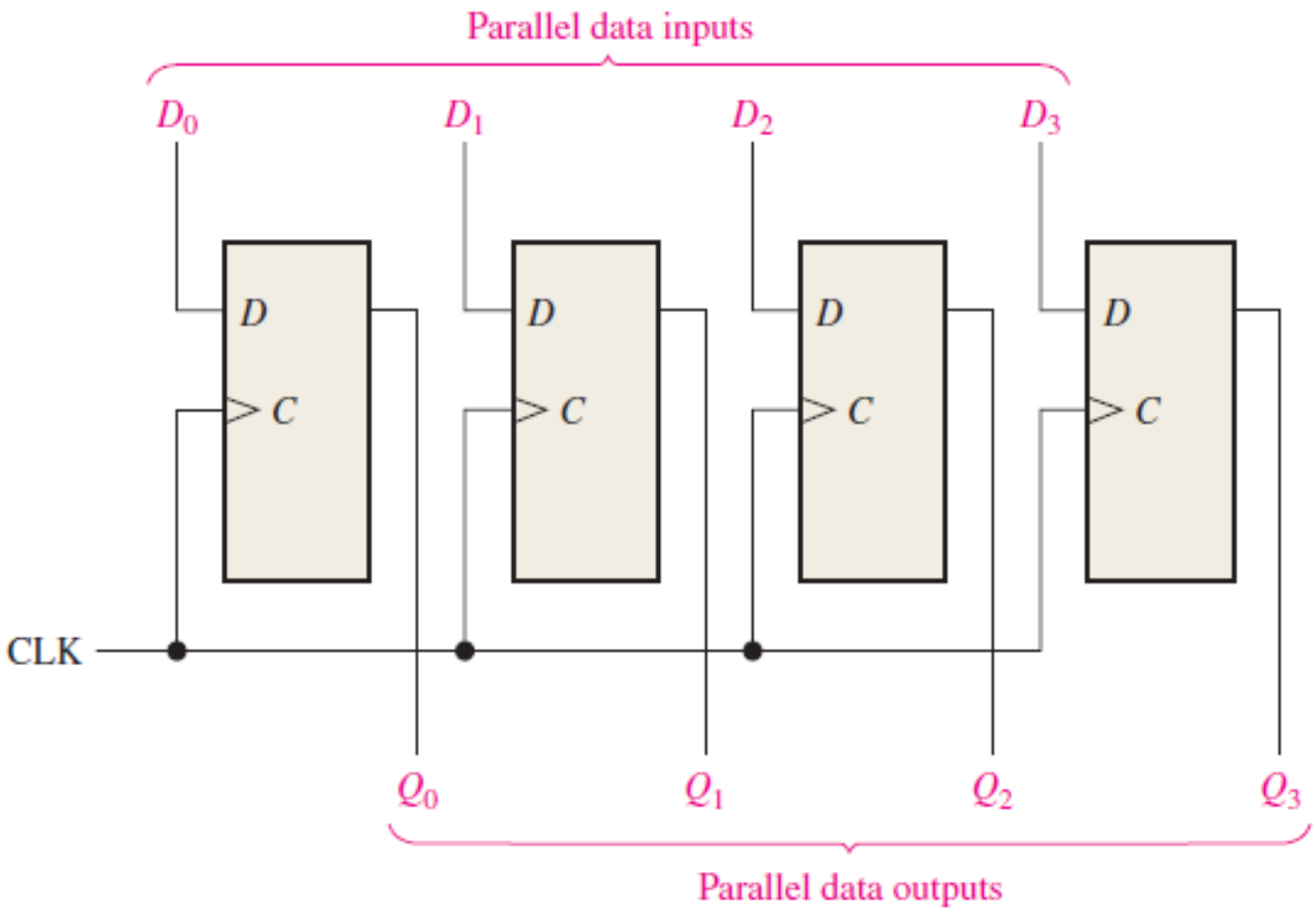
Parallel In/Serial Out Shift Registers



(b) Logic symbol

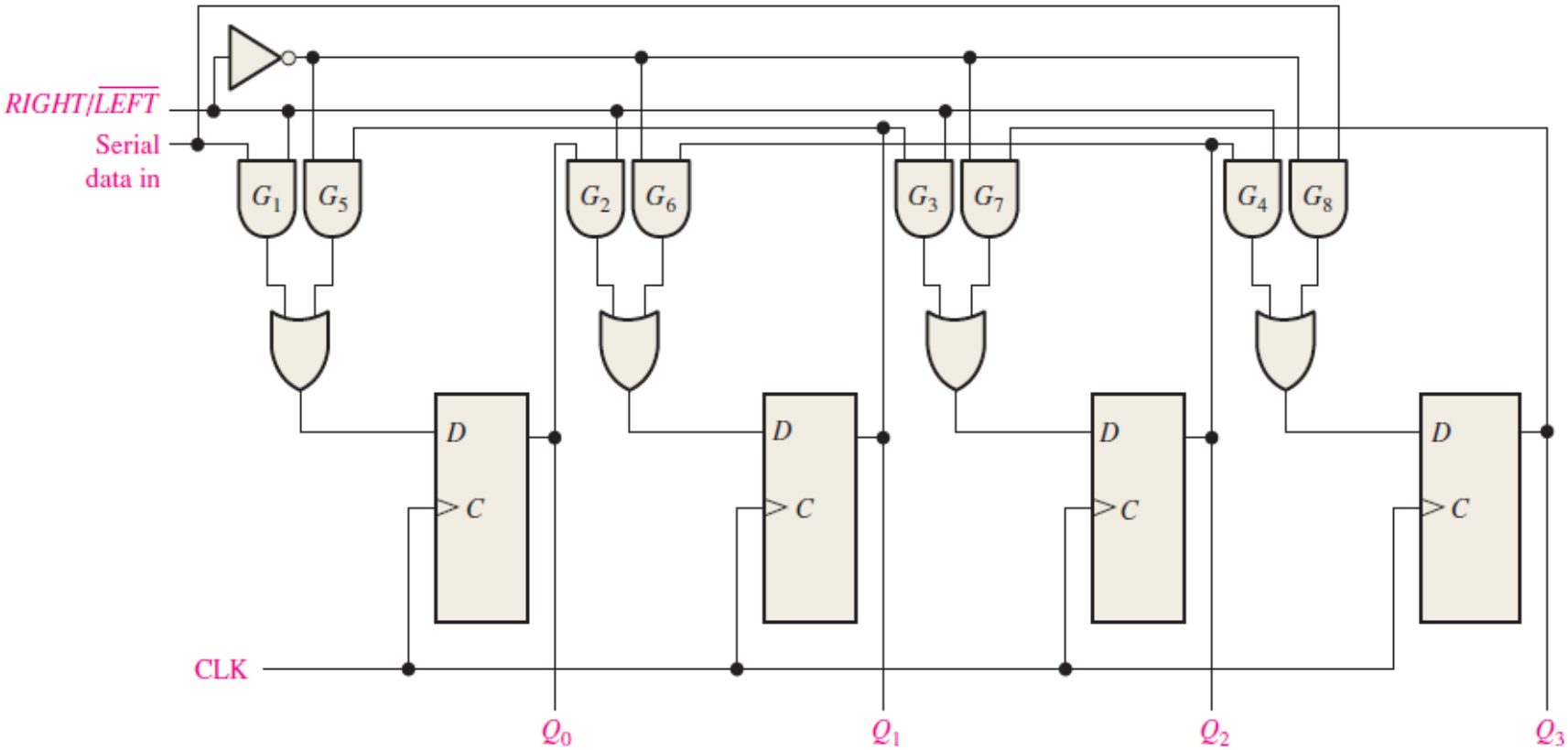
Parallel In/Parallel Out Shift Registers

- Simultaneous input and output of all data bits.



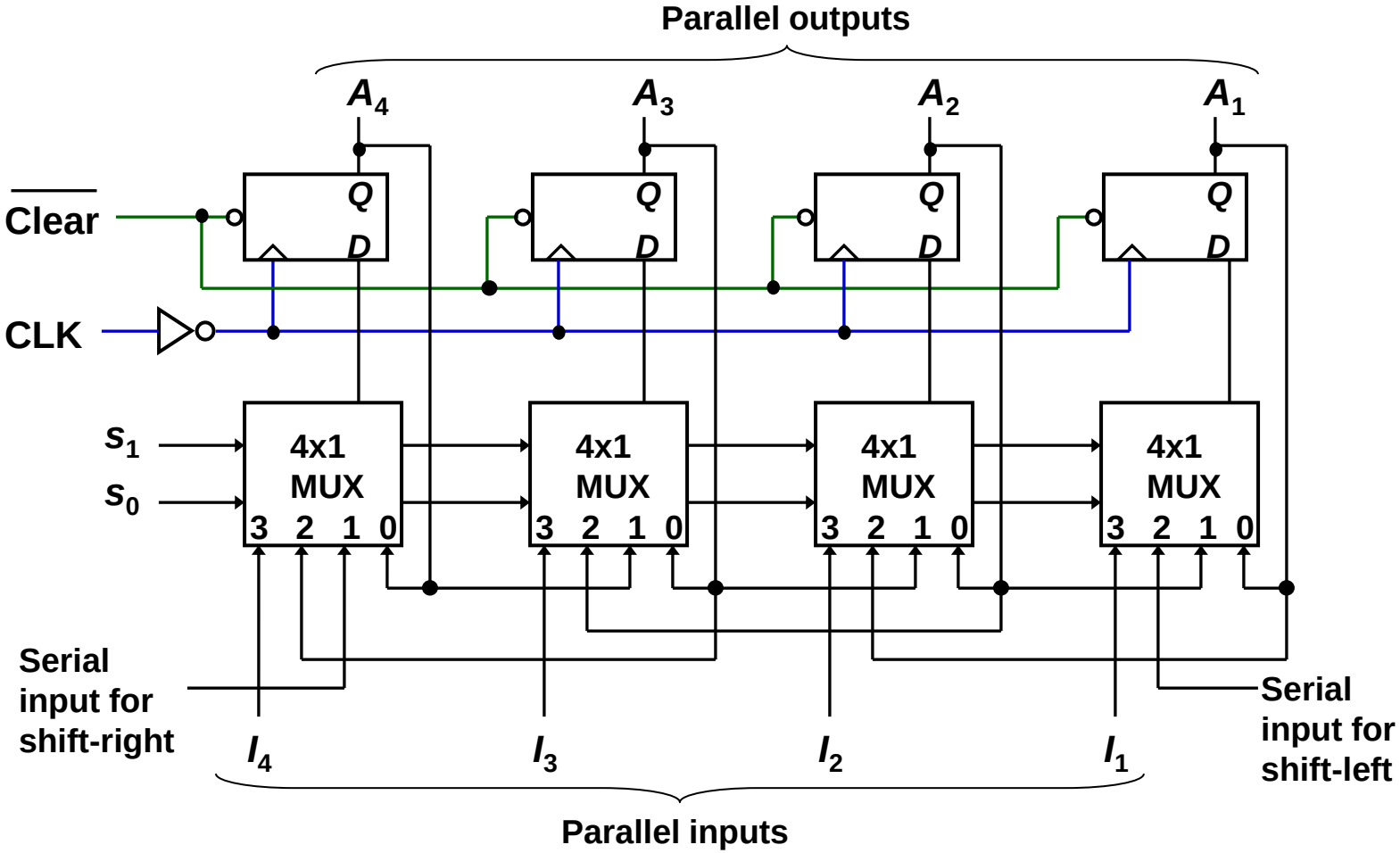
Bidirectional Shift Registers

- Data can be shifted either left or right, using a control line *RIGHT*/$\overline{LEFT}$ (or simply *RIGHT*) to indicate the direction.



Bidirectional Shift Registers

- 4-bit bidirectional shift register with parallel load.



Bidirectional Shift Registers

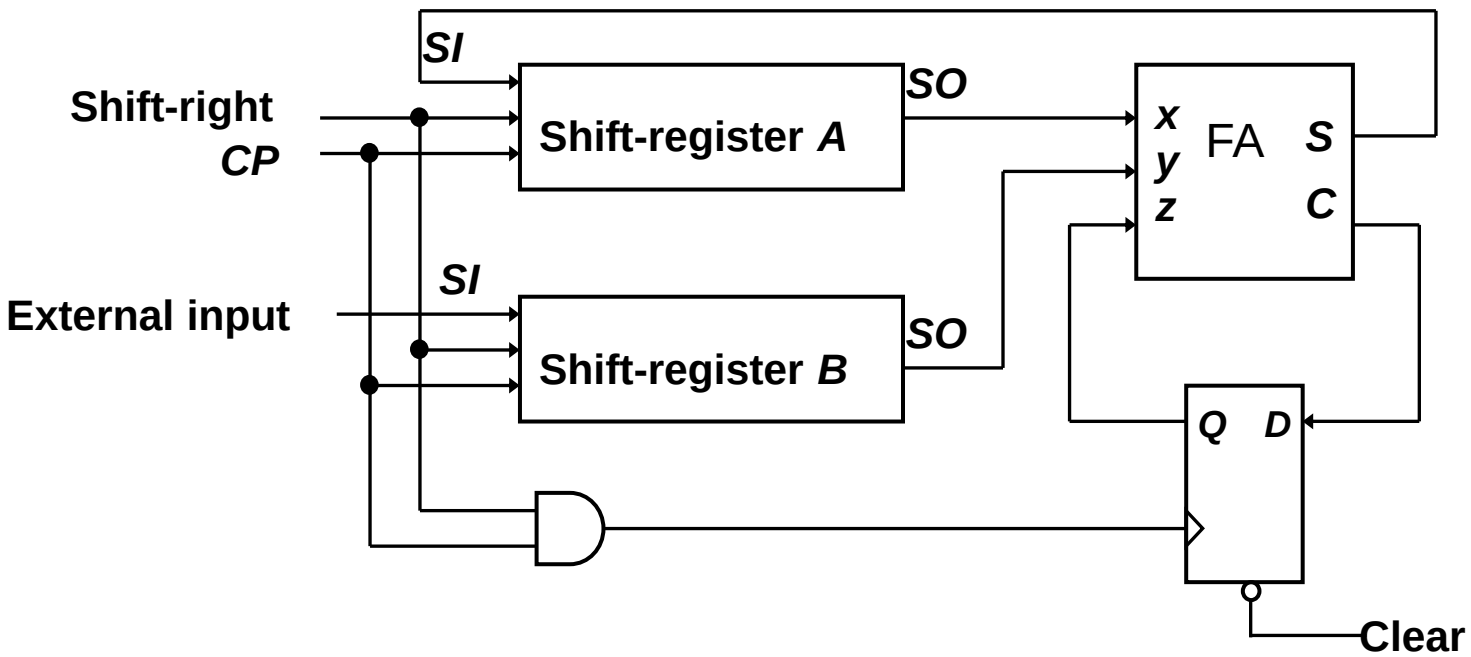
- 4-bit bidirectional shift register with parallel load.

Mode Control		
s_1	s_0	***Register Operation***
0	**0**	**No change**
0	**1**	**Shift right**
1	**0**	**Shift left**
1	**1**	**Parallel load**

An Application – Serial Addition

- Most operations in digital computers are done in parallel. Serial operations are slower but require less equipment.
- A serial adder is shown below.

$$A \leftarrow A + B.$$



An Application – Serial Addition

- *A* = 0100; *B* = 0111. *A* + *B* = 1011 is stored in *A* after 4 clock pulses.

Initial:		*A*: 0 1 0 0 *B*: 0 1 1 1	*Q*: 0
Step 1:	**0 + 1 + 0** **S = 1, C = 0**	*A*: 1 0 1 0 *B*: x 0 1 1	*Q*: 0
Step 2:	**0 + 1 + 0** **S = 1, C = 0**	*A*: 1 1 0 1 *B*: x x 0 1	*Q*: 0
Step 3:	**1 + 1 + 0** **S = 0, C = 1**	*A*: 0 1 1 0 *B*: x x x 0	*Q*: 1
Step 4:	**0 + 0 + 1** **S = 1, C = 0**	*A*: 1 0 1 1 *B*: x x x x	*Q*: 0

Shift Register Counters

- Shift register counter: a shift register with the serial output connected back to the serial input.

- They are classified as counters because they give a specified sequence of states.

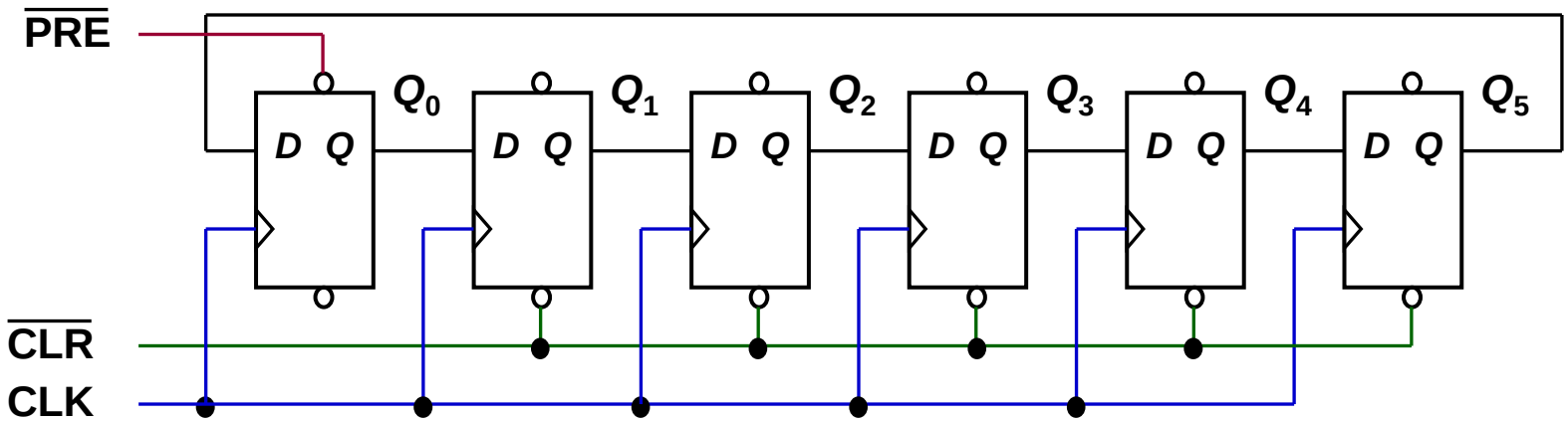
- Two common types: the Johnson counter and the Ring counter.

Ring Counters

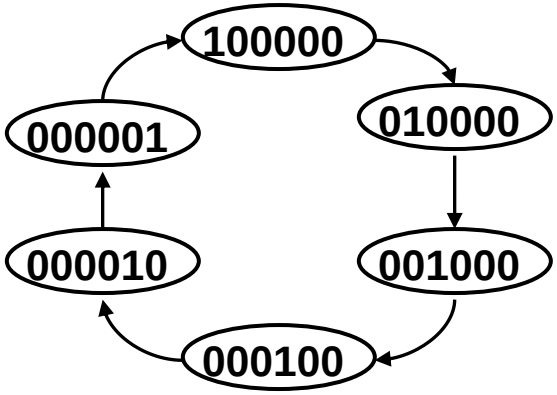
- One flip-flop (stage) for each state in the sequence.
- The output of the last stage is connected to the D input of the first stage.
- An *n-bit* ring counter cycles through *n* states.
- No decoding gates are required, as there is an output that corresponds to every state the counter is in.

Ring Counters

- Example: A 6-bit (MOD-6) ring counter.



Clock	Q_0	Q_1	Q_2	Q_3	Q_4	Q_5
0	1	0	0	0	0	0
1	0	1	0	0	0	0
2	0	0	1	0	0	0
3	0	0	0	1	0	0
4	0	0	0	0	1	0
5	0	0	0	0	0	1

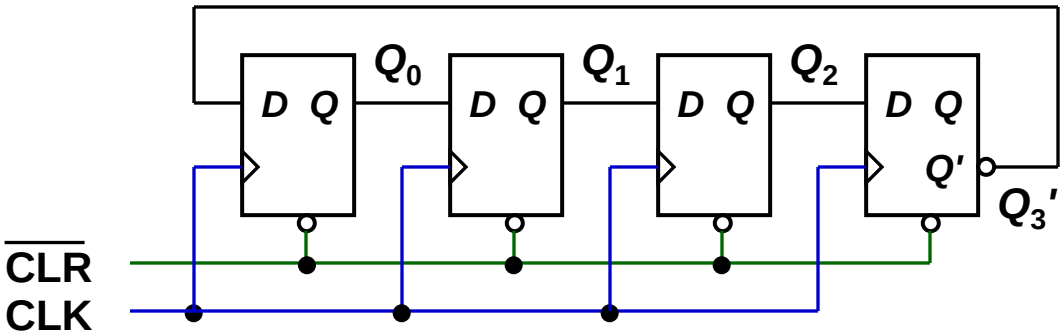


Johnson Counters

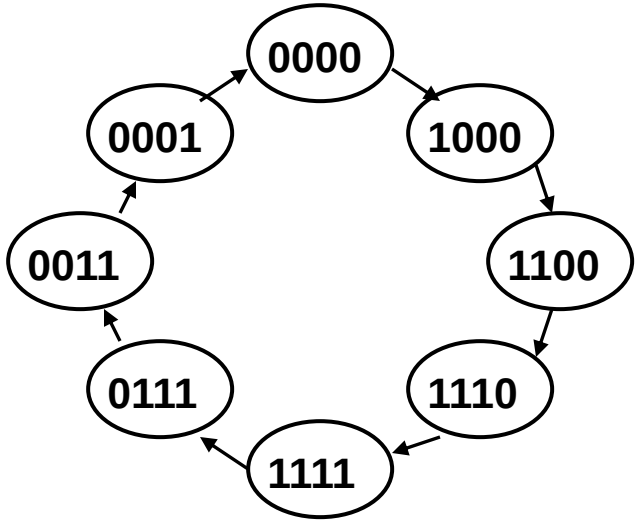
- The complement of the output of the last stage is connected back to the D input of the first stage.
- Also called the *twisted-ring counter*.
- Require fewer flip-flops than ring counters but more flip-flops than binary counters.
- An *n*-bit Johnson counter cycles through *2n* states.
- Require more decoding circuitry than ring counter but less than binary counters.

Johnson Counters

- Example: A 4-bit (MOD-8) Johnson counter.



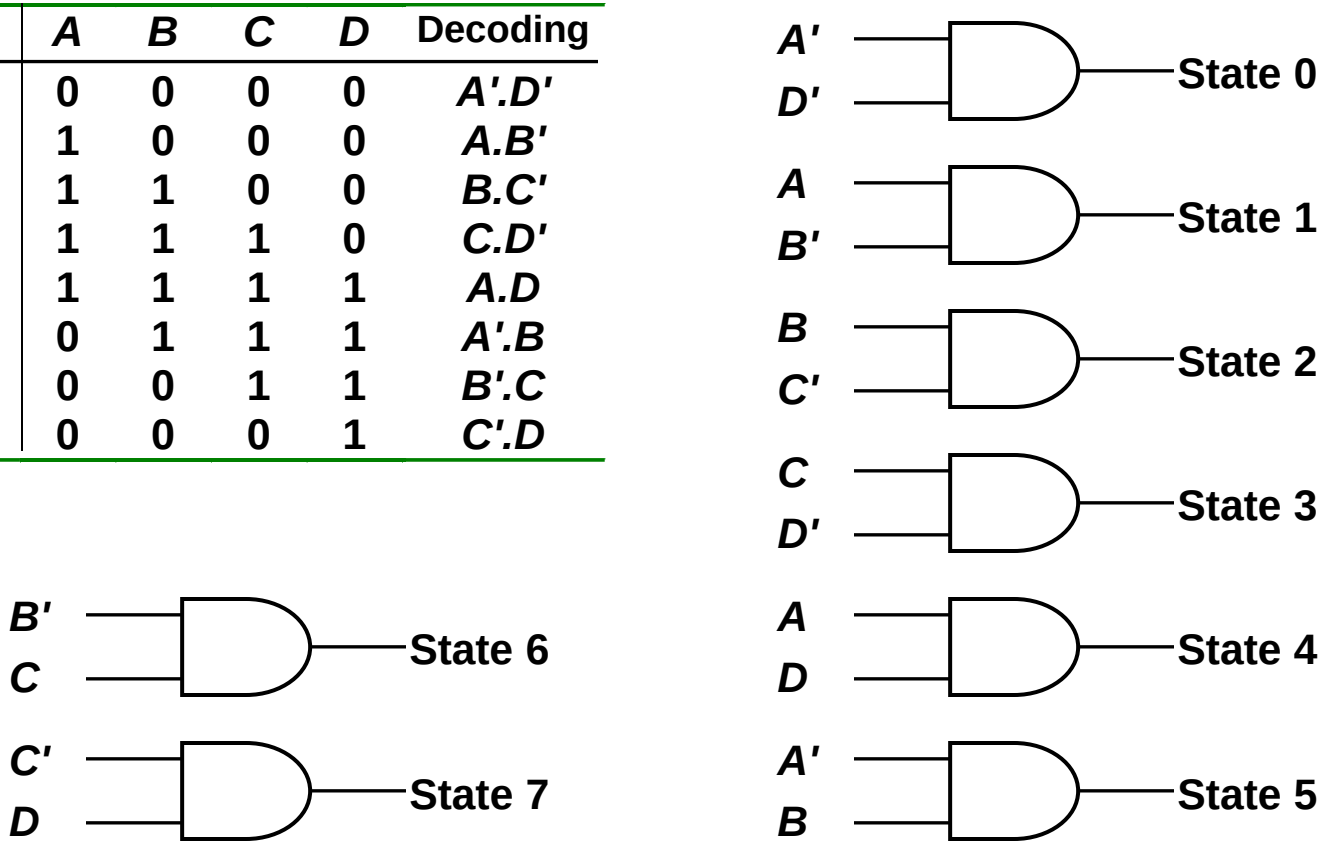
Clock	Q_0	Q_1	Q_2	Q_3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0
4	1	1	1	1
5	0	1	1	1
6	0	0	1	1
7	0	0	0	1



Johnson Counters

- Decoding logic for a 4-bit Johnson counter.

Clock	A	B	C	D	Decoding
0	0	0	0	0	A'.D'
1	1	0	0	0	A.B'
2	1	1	0	0	B.C'
3	1	1	1	0	C.D'
4	1	1	1	1	A.D
5	0	1	1	1	A'.B
6	0	0	1	1	B'.C
7	0	0	0	1	C'.D



References

- Chapter 8, Digital Fundamentals by Thomas L. Floyd

- Chapter 6, Digital Design by M. Morris Mano

- Disclaimer: I don’t own all the slides, these slides are copied and adopted from various public resources available on the internet.

Thank you!!!